The Good Plate: A Self-Stabilizing Adaptive Tray Realized as a Real-Time Cyber-Physical System

Zhen-Rong Wu
Dept. of Computer Science and Information Engineering
National Taiwan Normal University
41247012S@ntnu.edu.tw

Darrin Lin
Dept. of Computer Science and Information Engineering
National Taiwan Normal University
41247022S@ntnu.edu.tw

Xin-Shao Hon
Dept. of Computer Science and Information Engineering
National Taiwan Normal University
41247033S@ntnu.edu.tw

***Abstract*—We present *The Good Plate*, a self-stabilizing adaptive tray that keeps its top surface level despite external tilting disturbances, so that objects placed on it do not slide or topple. The platform is supported by a central universal joint and actuated by four MG90S servos that reel four strings attached to the platform corners. A GY-521/MPU6050 inertial measurement unit (IMU) senses the tilt, a complementary filter fuses its accelerometer and gyroscope into a drift-free attitude estimate, and two independent velocity-form proportional–integral (PI) controllers drive the pitch and roll angles toward zero. A key contribution is an exact inverse-kinematic model that converts a desired tilt angle into the precise string length each servo must produce, derived from first principles using a rigid-body rotation of the platform about the joint; this replaces the naive "servo-angle equals tilt-angle" mapping, which is strongly nonlinear and fails in practice. The firmware runs a closed loop at 500 Hz on an STM32F401 microcontroller. Using on-chip cycle-accurate timing across $36{,}031$ control cycles we show the task set is provably schedulable ($\sum_i C_i = 1629\,\mu\text{s} \le D = 2000\,\mu\text{s}$) with 0.00% deadline misses. Because our controller omits an explicit rate-damping term, the Lyapunov derivative cannot be signed analytically; we therefore certify stability *quantitatively from logged data* using two complementary measures—the exponential decay rate γ of the energy envelope and the fraction of samples for which $\dot{V} \le 0$. The data show a bounded, decaying energy ($\gamma = 0.275\,\text{s}^{-1}$) that settles into a $\sim 3.4^\circ$ residual ball, establishing *practical (ultimately bounded) stability*.**

***Index Terms*—cyber-physical systems, self-stabilization, complementary filter, PID control, inverse kinematics, real-time scheduling, Lyapunov stability, STM32, MPU6050.**

I. Introduction

Carrying a tray of objects—cups, plates, instruments—is a task humans perform by continuously adjusting the tray angle to keep it level. We set out to automate this reflex: an *adaptive tray* that, when tilted by an external disturbance (a bumped table, a vibrating surface, a hand carrying it unevenly), re-levels itself fast enough that objects resting on it do not slide off. This is a canonical cyber-physical system (CPS): a tight loop in which physical sensing, embedded computation, and physical actuation are co-designed and must meet real-time deadlines to be correct.

Rather than build a fully enclosed tray, we built the underlying *self-stabilizing platform*, which is the hard part. The platform rests on a single central universal joint that bears the load and allows two rotational degrees of freedom (pitch and roll). Four strings, attached to the four platform corners and wound on four servo spools, set the platform attitude by reeling string in or paying it out. The mechanism is shown in Fig. 1.

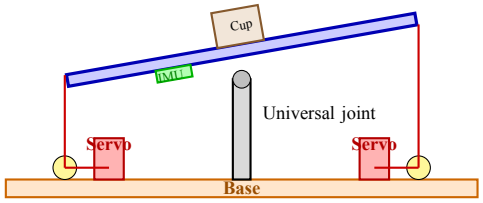

Fig. 1: Mechanical structure (one axis shown). The platform rests on a central universal joint that bears the load; four strings on servo-driven spools pull the four corners. Cross-pairs act antagonistically: one reels in while the opposite pays out. The IMU rides on the platform.

The contributions of this paper are:

1) A complete embedded CPS pipeline—IMU → complementary filter → velocity-form PI → string-length inverse kinematics → PWM—running closed-loop at 500 Hz on a single STM32F401.
2) An *exact* string-length model (Section V) derived from a rigid-body rotation about the joint, including the often-neglected upper-pillar offset, and its conversion to a servo angle through the spool circumference.
3) A formal and empirical real-time schedulability analysis based on on-chip cycle-counter measurements (Section VI).
4) A *data-driven* Lyapunov stability argument (Section VII) that, in the absence of an explicit damping term and an identified plant model, certifies practical stability through two quantitative measures, the decay rate γ and the $\dot{V} \le 0$ fraction.

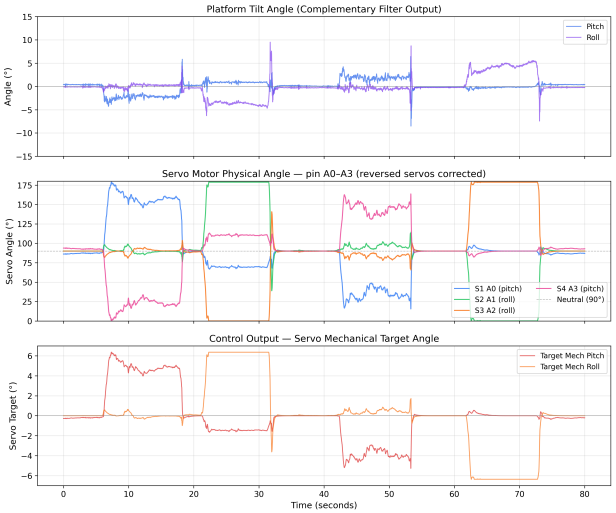

Fig. 2: Top-to-bottom: measured platform tilt (complementary-filter output), the four servos' physical angles (pin order A0–A3, with the two mechanically-reversed servos corrected), and the controller's accumulated mechanical target angle. The antagonistic action is visible: when one servo of a pair rises above the 90° neutral, its partner falls below it.

II. System Architecture

A. Hardware

The platform (Fig. 1) consists of:

- **MCU:** STM32F401CCU6 (Arm Cortex-M4F at 84 MHz).
- **IMU:** GY-521 module carrying an InvenSense MPU6050 (3-axis accelerometer + 3-axis gyroscope) on the platform, read over I^2C at 100 kHz.
- **Actuators:** $4\times$ MG90S hobby servos, each driving a spool that winds one string; PWM at 50 Hz from a single TIM2 with four compare channels.
- **Mechanism:** a central universal joint carries the platform load; the four strings only *steer* the attitude. Cross-pairs act *antagonistically*—to tilt one way, one string reels in while the opposite one pays out.

B. Signal flow

Each control cycle executes five tasks in a fixed sequence:

$$\underbrace{\text{IMU read}}_{\tau_1} \rightarrow \underbrace{\text{comp. filter}}_{\tau_2} \rightarrow \underbrace{\text{PI control}}_{\tau_3} \rightarrow \underbrace{\text{cable geometry}}_{\tau_4} \rightarrow \underbrace{\text{servo PWM}}_{\tau_5}.$$

Pitch and roll are controlled by two independent single-input single-output loops; all computation, logging (over USB-CDC), and timing instrumentation run on the STM32F401 within one 2 ms period.

III. Attitude Estimation

A. What the MPU6050 measures

The accelerometer measures *specific force* (in g): even at rest it reads $+1g$ along the axis opposing gravity, because the proof mass's spring reacts to gravity, $\vec{a}_{\text{meas}} = \vec{a}_{\text{motion}} - \vec{g}$. The gyroscope measures angular rate (in deg/s). Raw 16-bit samples are scaled by the configured sensitivities ($16384\,\text{LSB}/g$ at $\pm 2g$, $131\,\text{LSB}/(\text{deg/s})$ at ± 250 deg/s) and de-biased using offsets captured at startup, where the platform is held flat and still and 500 samples are averaged (the z-axis offset is referenced to $+1g$).

From the accelerometer alone, the instantaneous tilt angles are

$$\theta_p^{\text{acc}} = \operatorname{atan2}\big(a_y,\, a_z\big), \quad \theta_r^{\text{acc}} = -\operatorname{atan2}\Big(-a_x,\, \sqrt{a_y^2 + a_z^2}\Big). \tag{1}$$

B. Complementary filter

Neither sensor is sufficient alone: the accelerometer is accurate at low frequency (gravity is a stable reference) but is corrupted by inertial acceleration during motion; the gyroscope is accurate at high frequency but its integrated angle drifts over time. We fuse them with a fixed-weight *complementary filter* [1], [2], which high-passes the gyro and low-passes the accelerometer so the two transfer functions sum to unity gain:

$$\boxed{\hat{\theta}_k = \alpha\big(\hat{\theta}_{k-1} + \omega_k\,\Delta t\big) + (1-\alpha)\,\theta_k^{\text{acc}}} \tag{2}$$

with $\alpha = 0.96$ and Δt the measured loop period. The gyro axes map as gy $\rightarrow$ pitch and gx $\rightarrow$ roll. A Kalman filter would adapt the weights online, but the fixed-weight form is sufficient for our low-speed platform and costs only one multiply and one add per axis (Table I). We deliberately keep α high to favor the gyro short-term and let the accelerometer correct drift only slowly, which suppresses vibration artifacts.

IV. Control Design

A. Problem statement

With reference $\theta_{\text{ref}} = 0$, the error on each axis is $e(t) = \theta_{\text{ref}} - \theta(t) = -\theta(t)$. We want a control law that reacts quickly to disturbances, rejects steady offsets (e.g. when the whole base is tilted), and avoids overshoot. The classical answer is the PID controller [3], [4]:

$$u(t) = K_p\,e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d\,\frac{de(t)}{dt}. \tag{3}$$

B. Velocity (incremental) form

Discretizing with a rectangular integral and a backward-difference derivative and then differencing consecutive outputs, $u_k - u_{k-1}$, yields the *velocity form*:

$$\Delta u_k = K_p\,[e_k - e_{k-1}] + K_i\,\Delta t\,e_k + \frac{K_d}{\Delta t}\,[e_k - 2e_{k-1} + e_{k-2}]. \tag{4}$$

The second difference in the derivative term is extremely sensitive to sensor noise on a microcontroller. We therefore drop K_d and implement a **velocity-form PI**:

$$\boxed{\Delta u_k = K_P\,[e_k - e_{k-1}] + K_I\,e_k} \tag{5}$$

with $K_P = 0.1$, $K_I = 0.01$ in our firmware. The controller output is the *accumulated* target mechanical angle $u_k =$

$\sum_{j\le k}\Delta u_j$, which is exactly the integrator state. Accumulating (5) telescopes the first-difference term and recovers the positional law,

$$u_k = \sum_{j=0}^{k} \Delta u_j = K_P\, e_k + K_I \sum_{j=0}^{k} e_j, \tag{6}$$

so K_P plays the role of the *proportional* action and K_I that of the *integral* action—the velocity form is mathematically equivalent to a positional PI while being numerically gentler. To suppress residual noise on the first difference, $[e_k - e_{k-1}]$ is passed through a one-pole low-pass ($\beta = 0.85$) before use.

C. Practical safeguards

Four mechanisms make (5) robust on real hardware:

- **Soft deadband.** Errors below 0.3° are zeroed (with a continuous, offset-subtracting shaping so there is no jump at the edge), preventing servo chatter around level.
- **Leaky integrator.** A term $-\lambda\, u_k$ ($\lambda = 0.004$) is added to Δu_k, slowly pulling the accumulated command back toward zero. This bleeds off integrator wind-up and self-excitation at the cost of a small steady-state residual.
- **Adaptive slew limit.** The per-cycle step $|\Delta u_k|$ is clamped to a bound that grows with the error magnitude (0.12° for $< 4^\circ$ up to 0.6° for $> 20^\circ$), giving fast recovery for large disturbances and gentle, overshoot-free motion near level.
- **Conditional-integration anti-windup.** Before committing a step, the firmware computes the resulting servo command; if it would exceed the physical $[0^\circ, 180^\circ]$ range (or the $\pm 50^\circ$ mechanical tilt limit), the step is *rejected* and the integrator holds. Once the disturbance clears, the platform re-levels with no wind-up lag.

V. String-Length Geometry (Inverse Kinematics)

The PI controller outputs a desired tilt *angle*, but each servo controls the *length of a string*. A naive mapping—increment the servo angle by the angle error—fails because one degree of servo rotation does not produce one degree of platform tilt: the relationship is strongly nonlinear, since the strings connect a rotating platform to fixed servo positions. We instead compute the *exact* string length required at the desired tilt and convert that length to a servo rotation. The geometry (per axis) is shown in Fig. 3.

A. Step 1: locate the platform center

The platform pivots about the joint at $(0, b)$. A rigid *upper pillar* of length a connects the joint to the platform center, so the center rotates with the platform. At rest, the center sits at $(0, b + a)$; its offset from the joint is the vector $(0, a)^\top$. Rotating that offset by θ with the standard planar rotation matrix,

$$\text{offset}' = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} 0 \\ a \end{bmatrix} = \begin{bmatrix} -a\sin\theta \\ a\cos\theta \end{bmatrix}, \tag{7}$$

and adding back the joint position $(0, b)$ gives

$$\boxed{P_{\text{center}} = \big(-a\sin\theta,\ b + a\cos\theta\big).} \tag{8}$$

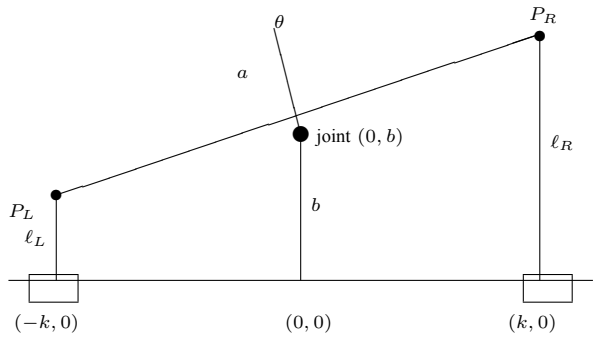

Fig. 3: Per-axis geometry. The platform tilts by θ about the universal joint at $(0, b)$. A rigid upper pillar of length a ties the joint to the platform center, so it rotates with the platform. Strings of length ℓ_L, ℓ_R run from the platform ends (offset $\pm k$) to fixed servos at $(\pm k, 0)$.

Keeping the a term is what distinguishes our model from a naive single-pillar setup: because a is not negligible relative to k, dropping it introduces a center-position error of up to $a \sin\theta_{\max}$.

B. Step 2: platform endpoints and string lengths

Each platform end is offset $\pm k$ *along* the platform. After the same rotation, those offsets become $(\pm k\cos\theta,\ \pm k\sin\theta)$. Adding them to P_{center}:

$$P_R = \big(-a\sin\theta + k\cos\theta,\ b + a\cos\theta + k\sin\theta\big), \tag{9}$$

$$P_L = \big(-a\sin\theta - k\cos\theta,\ b + a\cos\theta - k\sin\theta\big). \tag{10}$$

The string length is the Euclidean distance from each endpoint to its servo at $(\pm k, 0)$, $\ell_R = \|P_R - (k, 0)\|$, $\ell_L = \|P_L - (-k, 0)\|$, i.e.

$$\ell_R(\theta) = \sqrt{(-a\sin\theta + k\cos\theta - k)^2 + (b + a\cos\theta + k\sin\theta)^2}, \tag{11}$$

$$\ell_L(\theta) = \sqrt{(-a\sin\theta - k\cos\theta + k)^2 + (b + a\cos\theta - k\sin\theta)^2}. \tag{12}$$

These are exactly the expressions evaluated by the firmware routine `calculateTargets()`.

C. Sanity check at $\theta = 0$

Substituting $\sin 0 = 0,\ \cos 0 = 1$ into (11)–(12),

$$\ell_R(0) = \sqrt{(k-k)^2 + (b+a)^2} = a + b, \quad \ell_L(0) = a + b, \tag{13}$$

so both strings have the neutral length $a + b$ when level, as required. We work with the length *change* relative to neutral, $\Delta\ell_{R,L}(\theta) = \ell_{R,L}(\theta) - (a + b)$.

a) Numerical example: With the measured constants $a = 2.2$, $b = 4.5$, $k = 11.4$ (cm) and $\theta = 10^\circ$,

$$\ell_R(10^\circ) \approx 8.66\,\text{cm}\ (\Delta\ell_R \approx +1.96\,\text{cm, pay out}),$$
$$\ell_L(10^\circ) \approx 4.69\,\text{cm}\ (\Delta\ell_L \approx -2.01\,\text{cm, reel in}).$$

TABLE I: Measured Task Execution Times ($N = 36{,}031$ cycles)

	Task	Mean (μs)	P99 (μs)	WCET (μs)	Util.
τ_1	IMU read (I^{2}C)	1558	1558	1573	78.6%
τ_2	Complementary filter	6	6	7	0.4%
τ_3	PI control	2	3	3	0.1%
τ_4	Cable geometry	31	41	42	2.1%
τ_5	Servo PWM update	0	2	4	0.2%
	Total	1597	—	**1629**	**81.5%**

The two strings move *antagonistically*—one shortens while the other lengthens—which is precisely the behavior visible in Fig. 2.

D. Step 3: string length to servo angle

A servo spool of circumference C winds one full C of string per 360° revolution, so the servo rotation α that produces a length change $\Delta\ell$ is

$$\Delta\ell = C \cdot \frac{\alpha}{360^\circ} \implies \boxed{\alpha = \frac{360^\circ\,\Delta\ell}{C}}. \tag{14}$$

The firmware computes the *absolute* angle $\alpha = 360^\circ \ell / C$ for both the live tilt and (once, at boot) for $\theta = 0$, storing the latter as α_{base}. The PWM command for a standard 0–180° servo whose neutral is 90° is then

$$\theta_{\text{servo}} = 90^\circ + (\alpha - \alpha_{\text{base}}) = 90^\circ + \frac{360^\circ\,\Delta\ell}{C}, \tag{15}$$

with the sign inverted for the two mechanically-reversed servos. With $C = 5.1$ cm in our build, the $\Delta\ell_R = +1.96$ cm of the example maps to a servo rotation of $\alpha \approx 138^\circ$; a larger spool circumference trades range for finer resolution.

E. Full pipeline

Per axis and per cycle: (1) read IMU and estimate θ_p, θ_r via (2); (2) update the velocity-form PI (5) to get the target mechanical angles; (3) evaluate (11)–(12) for the required string lengths; (4) map each to a servo angle via (14); and (5) emit PWM on the four TIM2 channels. The same procedure runs for both axes within one 2 ms period.

VI. Real-Time Schedulability

A. Task model

The five tasks form a single-rate, non-preemptive sequential chain $\tau_1 \to \tau_2 \to \tau_3 \to \tau_4 \to \tau_5$ with a common control period and implicit deadline $T = D = 2000\,\mu$s (500 Hz). We instrumented each task with the Cortex-M4 Data Watchpoint and Trace (DWT) cycle counter [5] (at 84 MHz, 1 μs = 84 cycles) and logged 36,031 cycles. The per-task statistics are listed in Table I, and a representative timing diagram is shown in Fig. 4.

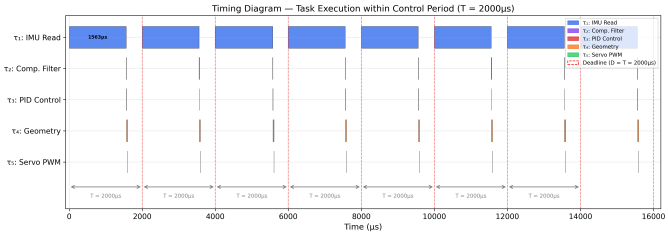

Fig. 4: Timing diagram over seven consecutive control periods. The IMU read τ_1 ($\sim$1563 μs) dominates; τ_2–τ_5 are negligible by comparison. Every period completes well before the $D = 2000\,\mu$s deadline.

B. Schedulability test

For a non-preemptive single-rate chain, all tasks must complete within one period, so the exact (necessary and sufficient) test is simply $\sum_i C_i \le D$:

$$1573 + 7 + 3 + 42 + 4 = 1629\,\mu\text{s} \;\le\; 2000\,\mu\text{s}, \tag{16}$$

leaving a slack of $D - \sum_i C_i = 371\,\mu$s (an 18.6% margin). Because execution is sequential, the worst-case response time of the chain equals $\sum_i C_i = 1629\,\mu\text{s} \le D$, so the system is **schedulable**.

a) On the Liu & Layland bound: The classical Liu & Layland utilization bound for $n = 5$ preemptive rate-monotonic tasks is $U \le n(2^{1/n} - 1) = 74.3\%$ [6]. Our measured $U = 81.5\%$ *exceeds* this bound, but the bound does not apply: it is a *sufficient* test for *preemptive* RM scheduling of independent periodic tasks, whereas our tasks are a single non-preemptive sequence sharing one period. For that model, $\sum_i C_i \le D$ is the correct, exact criterion [7], and it is satisfied.

C. Empirical verification

Across all 36,031 logged cycles the firmware recorded **0 deadline misses (0.00%)**, with observed loop WCET 1614 μs, mean 1597 μs, and P99 1608 μs—consistent with the per-task budget. The blocking I^{2}C read τ_1 consumes 78.6% of the budget and is the clear bottleneck; migrating the IMU to SPI ($\sim$400 μs) would free $\sim$1200 μs, enough for a substantially faster loop or a true rate-damping term.

VII. Stability Analysis

A. Lyapunov candidate and its derivative

Take the state $\mathbf{x} = [\theta_p, \theta_r]^\top$ with equilibrium $\mathbf{x} = \mathbf{0}$ (platform level), and the quadratic candidate

$$V(\mathbf{x}) = \tfrac{1}{2}\theta_p^2 + \tfrac{1}{2}\theta_r^2 = \tfrac{1}{2}\|\mathbf{x}\|^2. \tag{17}$$

V is the squared distance from level (analogous to spring energy $\frac{1}{2}kx^2$, not literal mechanical energy): $V = 0$ when flat, positive and radially unbounded otherwise. By the chain rule ($\frac{d}{dt}\theta^2 = 2\theta\dot\theta$),

$$\dot V = \theta_p\,\omega_p + \theta_r\,\omega_r, \tag{18}$$

where $\omega = \dot\theta$ is the gyro rate. Equation (18) has an intuitive reading: when the platform is tilted and *returning* (θ, ω of

opposite sign), $\dot{V} < 0$ and things are improving; when it is tilted and *overshooting* (same sign), $\dot{V} > 0$ and things are momentarily worsening.

B. Why $\dot{V}$ cannot be signed analytically

Lyapunov's direct method certifies asymptotic stability if $\dot{V} < 0$ for $\mathbf{x} \neq \mathbf{0}$ [8], [9]. We cannot establish this on paper for two reasons. First, our controller is *velocity-form PI*: the K_d term was deliberately dropped (Section V), so there is *no explicit rate-damping* that would inject a guaranteed $-c\,\omega^2$ into (18). Second, we have no identified plant model relating ω to θ (the string-driven, joint-supported mechanism with servo dynamics, friction, and string compliance is not modeled). Consequently $\dot{V} = \theta \cdot \omega$ is *not sign-definite*: it is strictly positive during every overshoot of an under-damped response. We therefore do *not* claim asymptotic stability analytically; instead we verify the decrease condition *quantitatively from logged data* using two complementary measures.

C. Measure 1: the $\dot{V} \leq 0$ fraction

We count, sample by sample, how often the energy is non-increasing:

$$\rho_{\Delta V} = \frac{1}{M-1} \sum_{k=0}^{M-2} \mathbf{1}\big[\, V[k{+}1] - V[k] \leq 0 \,\big] \times 100\%, \quad (19)$$

where $\mathbf{1}[\cdot]$ is the indicator function. Since $\dot{V} \approx (V[k{+}1] - V[k])/\Delta t$ with $\Delta t > 0$, we have $\operatorname{sign}(\Delta V) = \operatorname{sign}(\dot{V})$, so $\rho_{\Delta V}$ is exactly the fraction of time the Lyapunov decrease condition $\dot{V} \leq 0$ holds—it tests the *direction* of the energy change, not its speed.

a) Interpreting $\rho_{\Delta V} \approx 50\%$: Our data give $\rho_{\Delta V} = 49.7\%$ on average. This is *not* a sign of instability but the **signature of an under-damped oscillation**: energy *rises* during overshoot (about half the time) and *falls* during the return (about half), like a swing whose height goes up and down on each pass even as its peaks shrink. A per-sample count near 50% therefore tells us the system oscillates; whether it *converges* must be judged from the *envelope* of the peaks, which is what the second measure does.

D. Measure 2: the decay rate γ

The stronger, quantitative stability condition is that the energy decays proportionally to itself,

$$\dot{V} \leq -\gamma V, \qquad \gamma > 0. \quad (20)$$

Solving the equality case as a first-order linear ODE,

$$\frac{dV}{dt} = -\gamma V \;\Rightarrow\; \frac{dV}{V} = -\gamma\, dt \;\overset{\int}{\Longrightarrow}\; \ln V = -\gamma t + \text{const} \;\Rightarrow\; V(t) = V_0\, e^{-\gamma t}, \quad (21)$$

which is exponential decay with time constant $\tau = 1/\gamma$—the same form as a damped spring, an RC discharge, or Newtonian cooling, where V_0 is the energy at the onset of a disturbance. Taking logarithms linearizes it, $\ln V(t) = \ln V_0 - \gamma t$. We therefore extract γ per disturbance event by fitting a straight line to the *peak envelope* of $\ln V$ (a moving-max envelope, so noise valleys do not bias the slope) and reading

$$\gamma = -\,\text{slope of } \ln V_{\text{peak}} \text{ vs } t. \quad (22)$$

The interpretation is direct: $\gamma > 0$ means V decays (the envelope shrinks, the system converges); $\gamma = 0$ means a sustained oscillation; $\gamma < 0$ means V grows (unstable). The two measures are complementary: $\rho_{\Delta V}$ tests whether energy decreases *often*, while γ tests whether it decreases *on the whole*. An under-damped but stable system has $\rho_{\Delta V} \approx 50\%$ *and* $\gamma > 0$—precisely our case.

TABLE II: Per-Event Quantitative Stability Measures

Event	V_{peak}	γ (s^{-1})	τ (s)	$\rho_{\Delta V}$
#1	22.45	0.158	6.3	49.1%
#2	45.29	0.317	3.2	49.7%
#3	42.11	0.304	3.3	48.9%
#4	27.62	0.320	3.1	51.0%
avg	—	**0.275**	**3.6**	**49.7%**

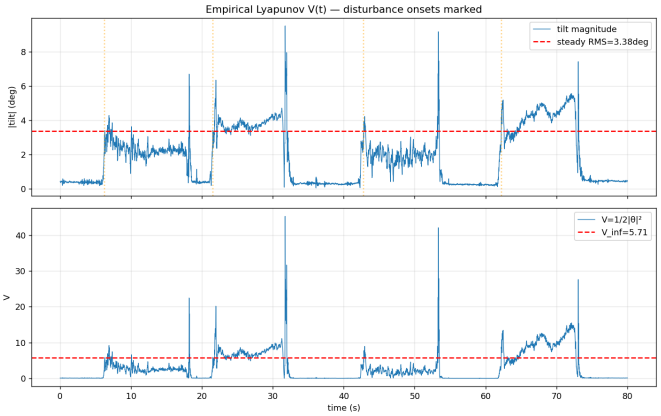

Fig. 5: Empirical Lyapunov function. Top: tilt magnitude $\|\mathbf{x}\|$ with the steady-state RMS line; dotted markers are disturbance onsets. Bottom: $V(t) = \frac{1}{2}\|\mathbf{x}\|^2$ with the ultimate-bound level $V_\infty = 5.71$. After each disturbance, V spikes and then decays back into the residual ball.

E. Results

We applied a quantitative analysis to the 80 s, 36,031-sample log, which contains four disturbance events (Fig. 5). The per-event results are in Table II. The energy is bounded throughout ($V_{\max} = 45.3 < \infty$, no escape), every event's envelope decays ($\gamma > 0$), and the average decay rate is $\gamma = 0.275\,\text{s}^{-1}$ ($\tau \approx 3.6\,\text{s}$). The $\dot{V} \leq 0$ fraction sits at 49.7%, confirming the under-damped-but-converging picture.

F. Ultimate bound and verdict

The trajectory does not converge to 0 but to a residual ball. From the last 20% of the log, the steady-state RMS tilt is 3.38° (max 7.43°), giving an ultimate bound $V_\infty = \frac{1}{2}(3.38^\circ)^2 = 5.71$, i.e. a ball of radius $\sim$3.4°. Combining the three facts—(i) V is bounded, (ii) the post-disturbance

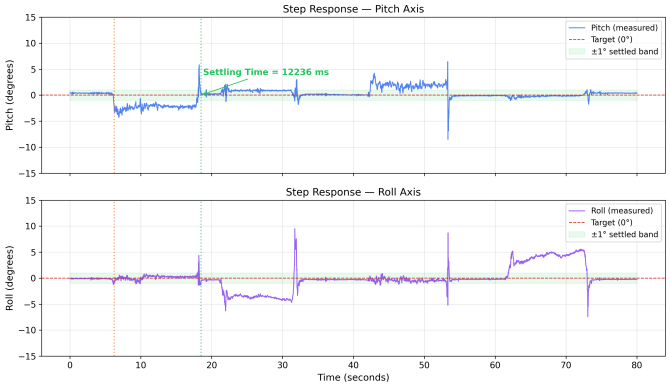

Fig. 6: Closed-loop step response (pitch and roll) across four disturbance events. The platform returns to the $\pm 1^\circ$ settled band after each push.

TABLE III: Performance Summary

Real-time	
Control frequency	500 Hz ($T = 2000\,\mu$s)
WCET ($\sum_i C_i$)	$1629\,\mu$s ($< 2000\,\mu$s)
CPU utilization	81.5%
Deadline margin	18.6%
Deadline miss rate	0.00% (0/36,031)
Control & stability	
Avg. settling time	11.96 s
Steady-state RMS tilt	3.38°
Lyapunov V	bounded ($V_{\max} = 45.3$)
Decay rate γ	$0.275\,\mathrm{s}^{-1}$ ($\tau \approx 3.6$ s)
Stability class	practical / ultimately bounded

envelope decays with $\gamma = 0.275\,\mathrm{s}^{-1}$, and (iii) the trajectory settles into a $\sim 3.4^\circ$ ball—the platform is **practically stable / ultimately bounded** [8], not asymptotically stable to 0. The nonzero residual is the expected price of the soft deadband, the leaky integrator, sensor noise, and—most importantly—the absence of an explicit velocity-damping term, which is exactly what the under-damped $\rho_{\Delta V} \approx 50\%$ signature predicts.

VIII. Experimental Results

Fig. 6 shows the closed-loop step response on both axes over the 80 s run. The platform absorbs peak disturbances of $\sim 8.2^\circ$ and recovers consistently to the $\pm 1^\circ$ band, with an average settling time of 11.96 s (a slow recovery that is a direct consequence of the missing damping term and the low PI gains chosen for vibration robustness). The steady-state RMS tilt is 3.38°. Table III consolidates the real-time and control metrics.

IX. Conclusion and Future Work

We built and verified a complete cyber-physical control loop—IMU $\rightarrow$ complementary filter $\rightarrow$ velocity-form PI $\rightarrow$ exact string-length inverse kinematics $\rightarrow$ PWM—for a self-stabilizing adaptive tray. The inverse-kinematic model, derived from a rigid-body rotation about the central joint, turns a desired tilt angle into the precise string length and servo rotation each actuator must produce, avoiding the nonlinear failure of a direct angle mapping. The task set is provably and empirically schedulable at 500 Hz ($1629 \leq 2000\,\mu$s, 0 deadline misses over 36,031 cycles). Because the controller has no explicit damping term and no identified plant model, we certified stability quantitatively from data: the Lyapunov energy is bounded and its post-disturbance envelope decays at $\gamma = 0.275\,\mathrm{s}^{-1}$, while the $\dot{V} \leq 0$ fraction near 50% correctly flags an under-damped—but converging—response that settles into a $\sim 3.4^\circ$ ball (practical / ultimate boundedness).

The analysis points directly to the next steps. Moving the IMU from $\mathrm{I}^2\mathrm{C}$ to SPI removes the dominant $\sim 1200\,\mu$s of τ_1, freeing budget for (i) a faster loop and (ii) a genuine gyro-rate damping term, which would inject the missing $-c\,\omega^2$ into $\dot{V}$, shrink the residual ball, and cut the settling time. Systematic K_P, K_I tuning and a Kalman attitude filter are natural further improvements.

Acknowledgment

The authors thank Prof. Chao Wang for guidance throughout the CSC9008 Cyber-Physical Systems course, and Chiin H. for assistance with the mechanical build.

References

[1] W. T. Higgins, "A comparison of complementary and kalman filtering," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-11, no. 3, pp. 321–325, 1975.

[2] S. Colton, "The balance filter: A simple solution for integrating accelerometer and gyroscope measurements for a balancing platform," in *Chief Delphi white paper*, 2007.

[3] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.

[4] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[5] Arm Ltd., "Cortex-M4 Technical Reference Manual (dwt cycle counter)," 2010.

[6] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[7] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York: Springer, 2011.

[8] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002.

[9] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Englewood Cliffs, NJ: Prentice Hall, 1991.